

EDA Final - Ruta Racso desde Cero

Solo problemas relevantes del grupo Utec CS3014 Projects

Que contiene este documento

Este documento queda limitado a los problemas del grupo de Codeforces que si sirven para el final segun la lista visible del curso:

- Proyecto 5
- Semana 13 - Clase 1 - Offline Dynamic Connectivity
- Semana 12 - Clase 1 - HLD
- Semana 11 - Problemas de strings
- Proyecto 4
- Semana 10 - Clase 1 - RMQ

Se quitaron Proyecto 3/BST implicito y Proyecto 2/Efecto cascada porque no son prioridad para el final actual. La meta no es practicar todo el curso, sino aprender desde cero lo que puede aparecer en el examen.

Orden de estudio si empiezas casi desde cero

1. **RMQ base**: aprende sparse table y por que responde minimo en $O(1)$.
2. **LCA/Euler tour**: entiende como convertir arboles en intervalos y como usar lca para caminos.
3. **Strings base**: suffix array, LCP, busqueda de patrones, subcadenas distintas y longest common substring.
4. **Tree queries**: DSU on tree, HLD, mascarar de paridad y centroid decomposition solo a nivel conceptual.
5. **Offline dynamic connectivity**: segment tree sobre tiempo + DSU con rollback.
6. **Proyectos 4 y 5**: usalos como integracion de distancias/caminos en arboles.

Mapa limpio de problemas

Tema	Contest	Problema	Prioridad
RMQ	694840/A	Static RMQ	Maxima
Strings	695758/A	Suffix array y LCP array	Maxima
Strings	695758/B	Busqueda de subcadenas	Maxima
Strings	695758/C	Contando subcadenas	Alta
Strings	695758/D	Subcadenas diferentes	Maxima
Strings	695758/E	Subcadena en comun mas larga	Maxima
Strings	695758/F	Ordenando subcadenas	Alta
Strings	695758/G	Bordes	Alta
Strings	695758/H	Refran	Alta
Strings	695758/I	Substring periodica	Media/alta
Arboles/HLD	697527/A	Tree and Queries	Maxima
Arboles/HLD	697527/B	Arpa letter-marked tree	Alta
Arboles/HLD	697527/C	Laminar Family	Media
Arboles/HLD	697527/D	Xenia and Tree	Alta
Arboles/HLD	697527/E	Beauty and The Tree	Media
Dynamic graph	698987/A	Connect and Disconnect	Maxima
Offline intervals	698987/B	Addition on Segments	Alta
Proyecto final	699336/A	Visitas	Alta
Distancias	694842/A	Distancia unitaria	Media

1 Semana 10 - Static RMQ

Problema A: Static RMQ

Link: <https://codeforces.com/group/Amk1RC81hZ/contest/694840/problem/A>

Que debes aprender primero

RMQ significa *Range Minimum Query*. Te dan un arreglo fijo A y muchas consultas $[l, r]$. Debes responder:

$$\min(A[l], A[l+1], \dots, A[r]).$$

La palabra clave es **static**: el arreglo no cambia. Si el arreglo cambia, necesitarías segment tree o Fenwick segun la operacion. Como no cambia, puedes preprocesar fuerte.

Solucion principal: sparse table

Define una tabla:

$$st[k][i] = \min(A[i], A[i+1], \dots, A[i+2^k-1]).$$

Es decir, guarda el minimo de cada bloque de longitud potencia de dos.

La transicion es:

$$st[k][i] = \min(st[k-1][i], st[k-1][i+2^{k-1}]).$$

El bloque grande de longitud 2^k se parte en dos bloques de longitud 2^{k-1} .

Para responder $[l, r]$, toma:

$$k = \lfloor \log_2(r-l+1) \rfloor.$$

Entonces dos bloques de longitud 2^k cubren todo el intervalo:

$$[l, l+2^k-1] \quad \text{y} \quad [r-2^k+1, r].$$

La respuesta es:

$$\min(st[k][l], st[k][r-2^k+1]).$$

Por que funciona

Los dos bloques pueden solaparse, pero eso no importa porque la operacion min es idempotente:

$$\min(x, x) = x.$$

Mientras ambos bloques cubran todo el rango, el minimo global esta en alguno de ellos. Si el minimo cae en la zona solapada, contarlos dos veces no cambia nada.

Complejidad

Preprocesamiento:

$$O(n \log n).$$

Consulta:

$$O(1).$$

Espacio:

$$O(n \log n).$$

Para teoria avanzada, 6.851 muestra RMQ en $O(n)$ espacio y $O(1)$ query usando bloques macro/micro, pero para resolver y explicar en examen, sparse table es la base que debes dominar.

2 Semana 11 - Strings

Base comun: suffix array y LCP

Un sufijo de S es una subcadena que empieza en una posición i y termina al final. El suffix array SA es el arreglo de posiciones de sufijos ordenados lexicográficamente. Si:

$$SA = [p_0, p_1, \dots, p_{n-1}],$$

entonces:

$$S[p_0..] < S[p_1..] < \dots < S[p_{n-1}..].$$

El arreglo LCP guarda cuanto prefijo comun tienen sufijos consecutivos:

$$LCP[i] = \text{lcp}(S[SA[i-1]..], S[SA[i]..]).$$

Esto sirve porque muchos problemas de strings se vuelven problemas de ordenar sufijos y medir prefijos comunes.

2.1 A. Suffix array y LCP array

Link: <https://codeforces.com/group/Amk1RC8lhZ/contest/695758/problem/A>

Que saber. Debes poder construir y explicar SA y LCP. El algoritmo de clase mas importante es Manber-Myers:

1. Primero ordenas sufijos por su primer caracter.
2. Luego por los primeros 2 caracteres.
3. Luego por los primeros 4.
4. Luego por 8, 16, etc.

En cada paso, el sufijo que empieza en i se representa por:

$$(rank[i], rank[i + 2^k]).$$

Si ordenas esos pares, ordenas por prefijos de longitud 2^{k+1} .

Correctitud. Si ya sabes el orden relativo de todas las subcadenas de longitud 2^k , entonces una subcadena de longitud 2^{k+1} queda determinada por sus dos mitades: primera mitad y segunda mitad. Por eso ordenar pares de ranks es suficiente.

Complejidad. Hay $O(\log n)$ rondas y cada ronda puede hacerse en $O(n)$ con counting sort/radix sort sobre ranks. Total:

$$O(n \log n).$$

2.2 B. Busqueda de subcadenas

Link: <https://codeforces.com/group/Amk1RC8lhZ/contest/695758/problem/B>

Idea. Para buscar un patron P dentro de un texto fijo S , construyes el suffix array de S . Todas las ocurrencias de P son sufijos que empiezan con P . En orden lexicografico, todos esos sufijos aparecen consecutivos.

Como se responde. Haces dos busquedas binarias:

- primera posición del suffix array cuyo sufijo es $\geq P$;
- primera posición cuyo sufijo es $> P$ como prefijo.

El intervalo entre ambas posiciones contiene todas las ocurrencias.

Por que no usar KMP siempre. KMP es excelente para un patron. Si tienes muchas consultas sobre el mismo texto, suffix array preprocesa el texto una vez y luego cada patron se busca rapido.

2.3 C. Contando subcadenas

Link: <https://codeforces.com/group/Amk1RC81hZ/contest/695758/problem/C>

Este problema entrena la idea de que las ocurrencias de un patron se cuentan por el tamano del intervalo de suffix array donde los sufijos tienen ese patron como prefijo. Si el intervalo es $[L, R]$, la cantidad de ocurrencias es:

$$R - L + 1.$$

Si la variante pide contar subcadenas con alguna propiedad, piensa primero:

- si depende de apariciones de un patron, usa intervalo en SA;
- si depende de subcadenas distintas, usa suma de LCP;
- si depende de repeticion, usa minimos en rangos de LCP.

2.4 D. Subcadenas diferentes

Link: <https://codeforces.com/group/Amk1RC81hZ/contest/695758/problem/D>

El numero total de subcadenas de un string de longitud n es:

$$\frac{n(n+1)}{2}.$$

Pero hay repetidas. Al ordenar sufijos, cada sufijo aporta sus prefijos. El sufijo $SA[i]$ aporta $n - SA[i]$ prefijos, pero los primeros $LCP[i]$ ya aparecieron en el sufijo anterior.

Formula:

$$\# \text{subcadenas distintas} = \frac{n(n+1)}{2} - \sum_i LCP[i].$$

Ejemplo mental. Si dos sufijos vecinos comparten prefijo de longitud 3, entonces las subcadenas de longitud 1, 2 y 3 que empiezan en el segundo sufijo ya fueron contadas por el primero.

2.5 E. Subcadena en comun mas larga

Link: <https://codeforces.com/group/Amk1RC81hZ/contest/695758/problem/E>

Dado A y B , construye:

$$T = A \# B \$,$$

donde $\#$ y $\$$ no aparecen en los strings. Construyes SA y LCP de T . La respuesta es el maximo $LCP[i]$ tal que los sufijos $SA[i-1]$ y $SA[i]$ vienen de strings distintos.

Por que basta revisar vecinos. Si una subcadena aparece en ambos strings, todos los sufijos que empiezan con esa subcadena quedan juntos en el suffix array. Dentro de ese bloque, si hay sufijos de ambos strings, existe algun par vecino de origen distinto.

2.6 F. Ordenando subcadenas

Link: <https://codeforces.com/group/Amk1RC81hZ/contest/695758/problem/F>

Para comparar dos subcadenas $S[l_1, r_1]$ y $S[l_2, r_2]$, necesitas saber su primer caracter distinto. Eso se obtiene con:

$$\text{lcp}(l_1, l_2).$$

Para obtener LCP entre sufijos arbitrarios, usas RMQ sobre el arreglo LCP: si los sufijos estan en posiciones a y b del suffix array, entonces:

$$\text{lcp}(a, b) = \min(LCP[a+1], \dots, LCP[b]).$$

Conclusion. Ordenar subcadenas requiere combinar strings con RMQ. Esta es una conexion muy probable para examen: suffix array + LCP + RMQ.

2.7 G. Bordes

Link: <https://codeforces.com/group/Amk1RC81hZ/contest/695758/problem/G>

Un borde es una cadena que es prefijo y sufijo a la vez. Para estudiar desde cero, usa KMP:

$$\pi[i] = \text{longitud del mayor borde de } S[0..i].$$

Los bordes de todo S se obtienen siguiendo:

$$k = \pi[n - 1], \quad k = \pi[k - 1], \quad \dots$$

hasta llegar a cero.

Relacion con suffix array. Tambien puedes verlo como LCP entre el sufijo que empieza en 0 y otros sufijos que terminan justo al final, pero KMP es mas simple para bordes.

2.8 H. Refran

Link: <https://codeforces.com/group/Amk1RC81hZ/contest/695758/problem/H>

Este tipo de problema suele buscar una subcadena repetida que maximiza una funcion como:

$$\text{longitud} \times \text{frecuencia}.$$

En suffix array, una subcadena aparece muchas veces si hay muchos sufijos consecutivos que comparten un prefijo largo. El arreglo LCP se interpreta como un histograma: un minimo alto en un bloque grande significa una subcadena larga que aparece muchas veces.

Estructura mental. Usa pila monotona sobre LCP para encontrar, para cada altura h , el bloque mas grande donde todos los LCP son al menos h . Eso da frecuencia y longitud.

2.9 I. Substring periodica

Link: <https://codeforces.com/group/Amk1RC81hZ/contest/695758/problem/I>

Una cadena tiene periodo p si:

$$S[i] = S[i + p]$$

siempre que ambas posiciones existan. Para substrings, necesitas verificar igualdades largas entre posiciones. Herramientas:

- Z-function o prefix-function para periodos globales;
- LCP entre sufijos para comparar substrings rapidamente;
- RMQ sobre LCP para LCP arbitrario.

Este es el mas avanzado del bloque. Si no sabes nada, primero asegura A–E.

3 Semana 12 - HLD y Tree Queries

Base: caminos y subarboles

En un arbol, hay dos tipos de preguntas muy comunes:

- sobre un subarbol;
- sobre el camino entre dos nodos.

Para subarboles, Euler tour convierte el subarbol de u en el intervalo $[\text{tin}(u), \text{tout}(u)]$. Para caminos, usas LCA/HLD.

3.1 A. Tree and Queries

Link: <https://codeforces.com/group/Amk1RC81hZ/contest/697527/problem/A>

Problema: cada nodo tiene color; una consulta pregunta en el subarbol de v cuantos colores aparecen al menos k veces.

Solucion. Usa DSU on tree. Conservas la informacion del hijo pesado y agregas los hijos ligeros solo cuando hace falta. Mantienes:

$$\text{cntColor}[c] = \text{apariciones del color } c,$$

y una estructura para saber cuantos colores tienen frecuencia al menos k .

Por que funciona. Antes de responder consultas de v , la bolsa contiene exactamente su subarbol. Los hijos son disjuntos; si agregas todos los hijos y el nodo, tienes el subarbol completo.

Complejidad. Cada nodo entra a bolsas asociadas a aristas ligeras $O(\log n)$ veces. La cota segura es $O(n \log n)$.

3.2 B. Arpa's letter-marked tree and Mehrdad's Dokhtar-kosh paths

Link: <https://codeforces.com/group/Amk1RC81hZ/contest/697527/problem/B>

La idea central es palindromos por paridad. Una multiconjunto de letras puede formar un palindromo si como maximo una letra aparece cantidad impar de veces. Representa paridades con una mascara de bits.

Para un camino, la mascara se obtiene con xor de prefijos desde la raiz. Dos mascaras son compatibles si:

- son iguales; o
- difieren en exactamente un bit.

Esto suele combinarse con DSU on tree para contar pares dentro de subarboles.

3.3 C. Laminar Family

Link: <https://codeforces.com/group/Amk1RC81hZ/contest/697527/problem/C>

Una familia laminar cumple que dos conjuntos cualesquiera son disjuntos o uno contiene al otro. En arboles, los subarboles tienen esta propiedad cuando se representan como intervalos Euler:

$$\text{subtree}(u) = [\text{tin}(u), \text{tout}(u)].$$

Dos intervalos de subarbol no se cruzan parcialmente: o son disjuntos o uno contiene al otro. La teoria importante es saber probar inclusion con tiempos:

$$u \text{ ancestro de } v \iff \text{tin}(u) \leq \text{tin}(v) \leq \text{tout}(u).$$

3.4 D. Xenia and Tree

Link: <https://codeforces.com/group/Amk1RC81hZ/contest/697527/problem/D>

Problema clasico: hay nodos rojos y consultas por distancia al rojo mas cercano. La estructura esperada es centroid decomposition.

Desde cero. Un centroide parte el arbol en componentes de tamaño a lo mucho la mitad. Si repites recursivamente, cada nodo tiene $O(\log n)$ centroides ancestros.

Operacion pintar. Para un nodo rojo v , actualizas cada centroide ancestro c con:

$$best[c] = \min(best[c], dist(v, c)).$$

Operacion preguntar. Para un nodo u , pruebas todos sus centroides ancestros:

$$\min_c best[c] + dist(u, c).$$

Como hay $O(\log n)$ centroides ancestros, cada operacion cuesta $O(\log n)$ si las distancias se obtienen con LCA o precomputacion.

3.5 E. Beauty and The Tree

Link: <https://codeforces.com/group/Amk1RC81hZ/contest/697527/problem/E>

Este problema es menos central que A–D. Entra como arbol + DP/probabilidad. Si aparece algo parecido en examen, la ruta mental es:

1. enraizar el arbol;
2. calcular tamanos de subarbol;
3. definir que informacion sube de hijos a padre;
4. si la respuesta depende de “ver el arbol desde todos los nodos”, usar rerooting.

4 Semana 13 - Offline Dynamic Connectivity

4.1 A. Connect and Disconnect

Link: <https://codeforces.com/group/Amk1RC81hZ/contest/698987/problem/A>

Este es core para dynamic graph. Hay operaciones que conectan y desconectan aristas, y consultas de conectividad. DSU normal no puede borrar aristas, por eso se procesa offline.

Paso 1: convertir aristas en intervalos de vida. Si una arista se agrega en tiempo l y se elimina en tiempo r , entonces vive en el intervalo $[l, r)$. Si nunca se elimina, vive hasta el final.

Paso 2: segment tree sobre tiempo. Cada intervalo de vida se descompone en $O(\log q)$ nodos de un segment tree que cubre los tiempos.

Paso 3: DFS con DSU rollback. Al entrar a un nodo del segment tree, agregas todas sus aristas al DSU. Al salir, deshaces esas uniones. En una hoja, el DSU contiene exactamente las aristas activas en ese instante.

Por que rollback. Path compression hace difícil deshacer cambios. Por eso DSU rollback usa union by size/rank, guarda cambios en una pila, y revierte al salir del intervalo.

Complejidad. Cada arista aparece en $O(\log q)$ nodos. Cada union/find cuesta $O(\log n)$ sin compresion o casi constante en la practica con union by size. La cota segura:

$$O((q + m) \log q \log n).$$

4.2 B. Addition on Segments

Link: <https://codeforces.com/group/Amk1RC81hZ/contest/698987/problem/B>

Aunque no sea grafo, entrena la misma idea: una operacion existe en un intervalo. Cuando veas “agregar algo en un segmento de tiempo” o “activar/desactivar”, piensa en segment tree sobre tiempo.

La leccion importante para el examen:

- si hay operaciones que se activan y desactivan;
- y puedes leer todo el input antes de responder;
- entonces convierte cada objeto en intervalos de vida.

5 Proyecto 5 y Proyecto 4

5.1 Proyecto 5: Visitas

Link: <https://codeforces.com/group/Amk1RC81hZ/contest/699336/problem/A>

Por estar al final del bloque y por el nombre, debe tratarse como problema de arboles/caminos/visitas. Si es una variante tipo consultas sobre caminos, las herramientas probables son:

- LCA para descomponer caminos;
- diferencias en arbol si las visitas son offline;
- HLD si hay updates/queries sobre caminos;
- DSU on tree si las visitas son por subarbol/color.

Como no debes aprender todo, la regla practica es:

1. Si preguntan cantidad de rutas que pasan por aristas: diferencia en arbol.
2. Si preguntan max/min/suma en camino con cambios: HLD.
3. Si preguntan colores en subarbol: DSU on tree.
4. Si preguntan conectividad con cambios: offline dynamic connectivity.

5.2 Proyecto 4: Distancia unitaria

Link: <https://codeforces.com/group/Amk1RC81hZ/contest/694842/problem/A>

Si es de distancias en arbol, la formula que debes memorizar es:

$$dist(u, v) = depth[u] + depth[v] - 2depth[lca(u, v)].$$

Si es de grafo no ponderado, la palabra “unitaria” sugiere BFS porque todas las aristas pesan 1. Para el final, este problema vale como practica secundaria: distancia en arbol/grafos, no como tema principal.

6 Problemas externos utiles que si se conservan

Estos problemas no son Proyecto 2/BST ni Efecto cascada; si sirven porque entrenan exactamente los temas finales: RMQ/LCA, strings y dynamic graph.

6.1 Imperial roads

Link: <https://codeforces.com/gym/101889/problem/I>

Tema: MST + LCA/RMQ. Construyes un MST con peso total W . Para una arista forzada (u, v, w) , al agregarla al MST aparece un ciclo formado por esa arista y el camino entre u y v en el MST. Para mantener un arbol que incluya la arista forzada, eliminas la arista mas pesada del camino. Respuesta:

$$W + w - \maxEdge(u, v).$$

La estructura de consulta es LCA con maximo en saltos binarios, o HLD con segment tree de maximos.

6.2 Fools and Roads

Link: <https://codeforces.com/problemset/problem/191/C>

Tema: LCA + diferencia en arbol. Para cada ruta (u, v) :

$$\delta[u] + \delta[v] - \delta[lca(u, v)] = 2.$$

Luego haces DFS postorden. El acumulado de un hijo indica cuantas rutas pasan por la arista entre ese hijo y su padre. Es mejor que HLD porque todas las rutas se conocen offline y solo se necesita contar al final.

6.3 Frequency of String

Link: <https://codeforces.com/problemset/problem/963/D>

Tema: strings y ventanas de ocurrencias. Para un patron P , si sus ocurrencias en el texto empiezan en:

$$p_1 < p_2 < \dots < p_t,$$

la minima subcadena que contiene k ocurrencias se obtiene con:

$$\min_i (p_{i+k-1} - p_i + |P|).$$

El reto es obtener las listas de ocurrencias de muchos patrones. Las herramientas naturales son Aho-Corasick para muchos patrones, o suffix array si se razona por intervalos lexicograficos.

6.4 Yosupo - Dynamic Graph Vertex Add Component Sum

Link: https://judge.yosupo.jp/problem/dynamic_graph_vertex_add_component_sum

Tema: dynamic connectivity offline con suma de componente. Cada arista tiene intervalos de vida. Se usa segment tree sobre tiempo y DSU rollback. El DSU guarda tambien suma de pesos por componente. En cada hoja temporal, la estructura representa exactamente el grafo activo en ese momento.

6.5 Yosupo - Dynamic Tree Subtree Add Subtree Sum

Link: https://judge.yosupo.jp/problem/dynamic_tree_subtree_add_subtree_sum

Tema: Euler-tour trees. Un arbol dinamico se representa como una secuencia Euler guardada en un BST balanceado. Link y cut son operaciones de split/merge. Sumar en subarbol y consultar suma de subarbol son operaciones de intervalo con lazy propagation.

6.6 SPOJ - Dynamic Tree Connectivity

Link: <https://www.spoj.com/problems/DYNACON1/>

Tema: conectividad dinamica en bosques. Online se resuelve con Link-Cut Trees o Euler-Tour Trees. Offline se puede resolver con segment tree sobre tiempo y DSU rollback. Es buen puente entre dynamic tree y dynamic graph.

7 Resumen final: que estudiar y que ignorar

Estudia

- Static RMQ.
- Suffix array, LCP, busqueda de patrones, subcadenas distintas, LCS, bordes y periodicidad.
- DSU on tree, Euler tour, LCA, mascarar xor en arboles.
- Centroid decomposition para Xenia and Tree a nivel de idea.
- Offline dynamic connectivity con segment tree sobre tiempo y DSU rollback.
- Imperial roads, Fools and Roads, Frequency of String, Yosupo dynamic graph/tree y SPOJ Dynamic Tree Connectivity.
- Proyecto 5 y Proyecto 4 solo como integracion de arboles/distancias.

Ignora para esta ruta

- Proyecto 3 - BST implicito.
- Proyecto 2 - Efecto cascada.
- Cualquier otro problema externo que no sea Imperial, Fools and Roads, Frequency of String, Yosupo dynamic graph/tree o SPOJ Dynamic Tree Connectivity, salvo que el profesor lo mencione directamente.

Checklist para resolver en papel

1. Identifica si es string, arbol estatico, camino en arbol, subarbol, o grafo dinamico.
2. Escribe la estructura: sparse table, suffix array, DSU on tree, HLD, centroid decomposition, o DSU rollback.
3. Escribe el invariante: que guarda la estructura y por que representa el problema.
4. Prueba correctitud en 5-8 lineas.
5. Cierra con complejidad de preprocesamiento, query, update y memoria.